Hermes with General Agents

Personal Agent Harnesses, Chapter 2/2 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/hermes

EN: Hermes is your self-improving AI Employee -- Nous Research's open-source agent running on your own infrastructure. OpenClaw bet on breadth (every channel); Hermes bets on depth (learn, compound).
Roman Urdu: Hermes aapka self-improving AI Employee hai -- Nous Research ka open-source agent jo aapki apni infrastructure pe chalta. OpenClaw ne breadth pe bet lagai (har channel); Hermes ne depth pe (seekho, compound karo).

1. OpenClaw vs Hermes

- OpenClaw: breadth bet, you deliberately commit memory, you install skills, model is fixed. Hermes: depth bet, it commits memory itself (no wall), it WRITES its own skills, model is swappable with no lock-in.
- Honest ceiling: memory/skills are curated, the model itself is never retrained -- the real risk is QUIET DRIFT, hence the monthly audit.

2. Install + the Closed Learning Loop

- 4 risks before install: runaway spend, prompt injection, skills supply-chain risk, destructive actions/leaks -- each needs a guardrail (spending limits, draft-over-send, source review+version pin, secrets in .env never in chat).
- **Closed learning loop:** after a substantial task, Hermes decides whether to save memory or write a new skill in ~/.hermes/skills/. If it doesn't act, say: 'save that as the way you want this done.'

3. Memory, Model-Swap, Automation

- Hermes curates memory itself, recalls across sessions (even after /new) -- convenient, but needs periodic review of memories/ (vs. OpenClaw's auditable, hand-written approach).
- Model swap is one command -- the durable asset is the skill+memory layer, the brain is replaceable. Caution: not every model is equal; optimize skills for smaller models.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Closed learning loop	After a substantial task, Hermes decides to save memory or write a new skill.	Substantial task ke baad, Hermes memory save ya naya skill likhne ka faisla karta.
Quiet drift	The real risk of a self-improving agent -- not retraining, but slow uncontrolled change.	Self-improving agent ka asal risk -- retraining nahi, slow uncontrolled change.
Model swap	Switching Hermes's underlying model with one command, no lock-in.	Hermes ka underlying model ek command se switch karna, lock-in ke bagair.
7-rung ladder	Download & go -> knows you -> commands & model-agnostic -> integrator -> orchestration -> builder -> one operating system.	Download & go -> knows you -> commands & model-agnostic -> integrator -> orchestration -> builder -> one operating system.
3 complementary open layers	OpenClaw (gateway/breadth), Hermes (learner/depth), Paperclip (orchestrator).	OpenClaw (gateway/breadth), Hermes (learner/depth), Paperclip (orchestrator).

Quick Self-Check

Question	Answer
What's the core bet-difference between OpenClaw and Hermes?	OpenClaw bets on breadth (reach via every channel); Hermes bets on depth (learning and compounding over time).
What's the 'closed learning loop' Hermes runs after a substantial task?	It decides whether to save a memory or write a new skill file -- if it doesn't, you can explicitly tell it to save the approach.
Why is 'self-improving' an honest but limited claim?	Memory and skills get curated, but the underlying model itself is never retrained -- improvement is additive, not deep learning.
What's the real risk this chapter names for a self-improving agent, and its defense?	Quiet drift -- defended by a monthly audit of skills, memory, and the supply chain.

Where This Leads / Sources

Leads to: Governance Risk & Responsible Use, Is This an Agent Problem?. Full 7+ scenarios in the chapter's numbered files.

Source: docs/hermes/ (this repo), content originally via Zia Tutor AI connector.